Programowanie obiektowe

Krzysztof Dmitruk

Uniwersytet Marii Curie-Skłodowskiej w Lublinie

2026

Zespół

Wykład:

dr Krzysztof Dmitruk

krzysztof.dmitruk@umcs.pl

konsultacje: środa 12:00-14:00

Laboratorium:

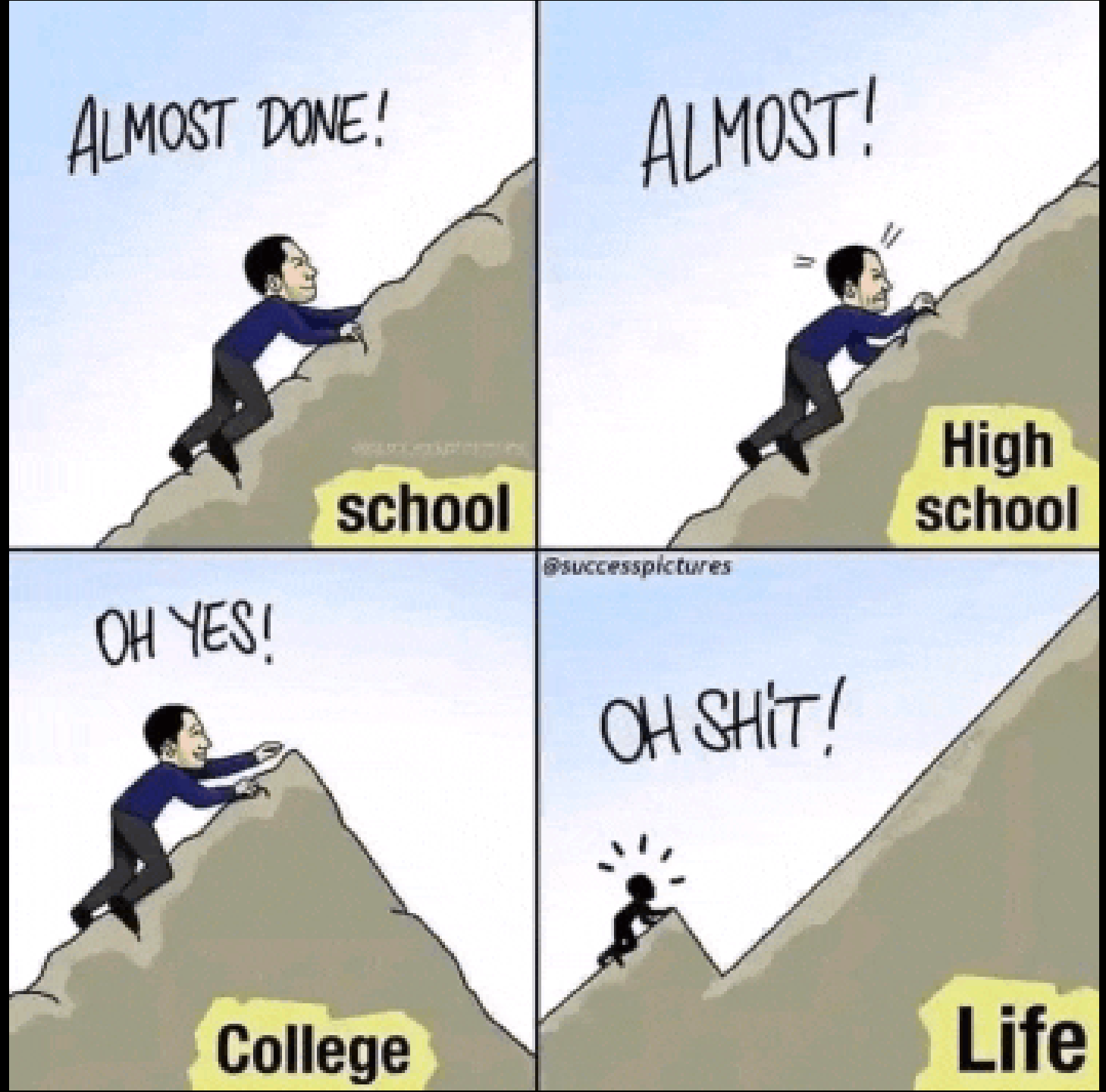
mgr inż. Emilia Bober
mgr Bartłomiej Daniluk
dr inż. Marcin Kurzyna
mgr Rafał Niedziółka-Domański
mgr Maciej Okoń
dr Krzysztof Dmitruk

Programowanie na studiach

Podstawy programowania

Programowanie obiektowe

Inżynieria oprogramowania


ALMOST DONE!
school
ALMOST!
High school
OH YES!
College
@successpictures
OH SHIT!
Life

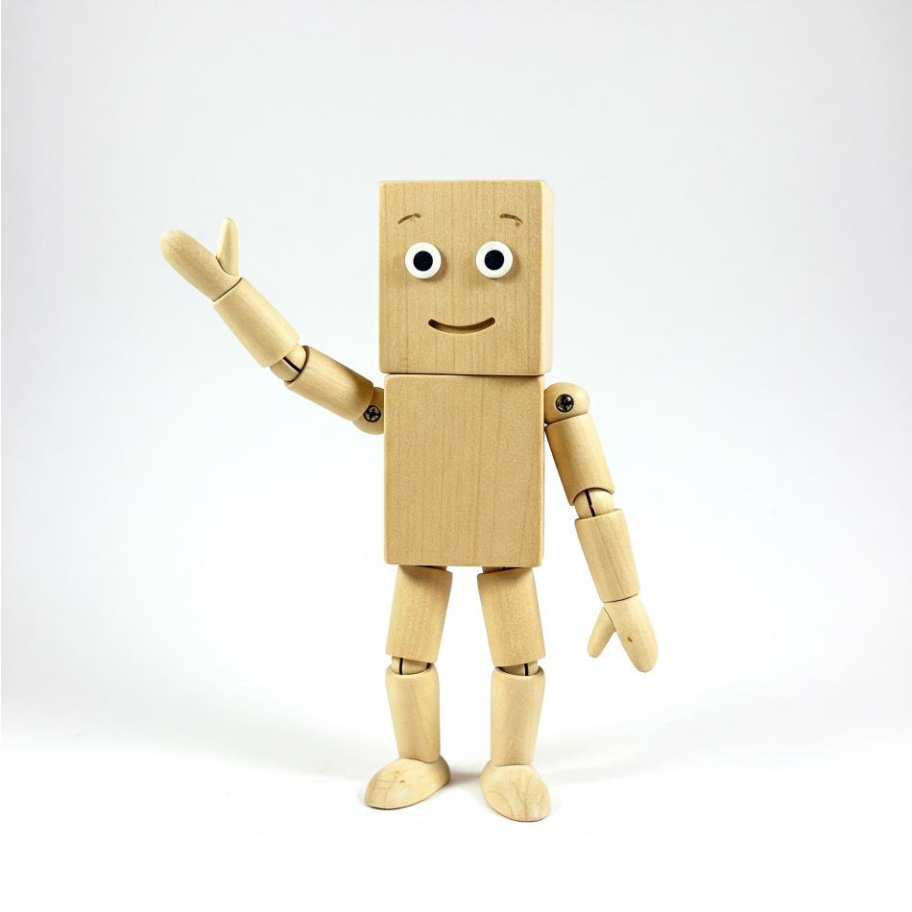
Plan przedmiotu

- Podstawy programowania obiektowego i Javy:
  - Wykłady 1-8,
  - Kolokwium 1.

- Elementy programowania użytkowego:
  - Wykłady 9-13,
  - Kolokwium 2.

- Temat dodatkowy.

Wykład 1. Wstęp do obiektowości

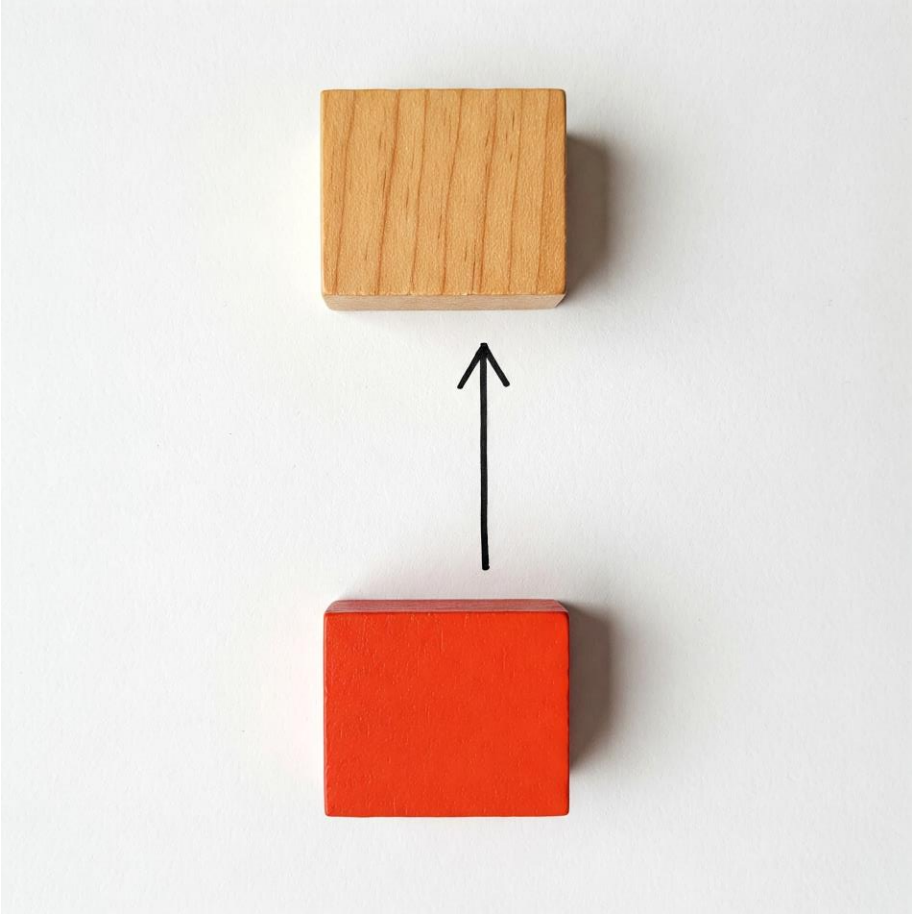


- Klasy i obiekty
- Wstęp do Javy

Wykład 2. Dostęp do danych

- Abstrakcja
- Hermetyzacja
- Specyfikatory dostępu
- Konstruktory
- Reprezentacja zmiennych w pamięci

Wykład 3. Powiązania między klasami



- Asocjacje
- Dziedziczenie
- Schemat klas UML

Wykład 4. Kontenery

- Interfejsy
- Kontenery:
  - Tablica
  - Lista
  - Zbiór
  - Stos
  - Mapa

Wykład 5. Polimorfizm

- Polimorfizm:
  - statyczny
  - dynamiczny
- Wirtualność
- Wzorce projektowe

Wykład 6. Serializacja

- Obsługa plików
- Serializacja danych
- Wyrażenia regularne
- Wyjątki

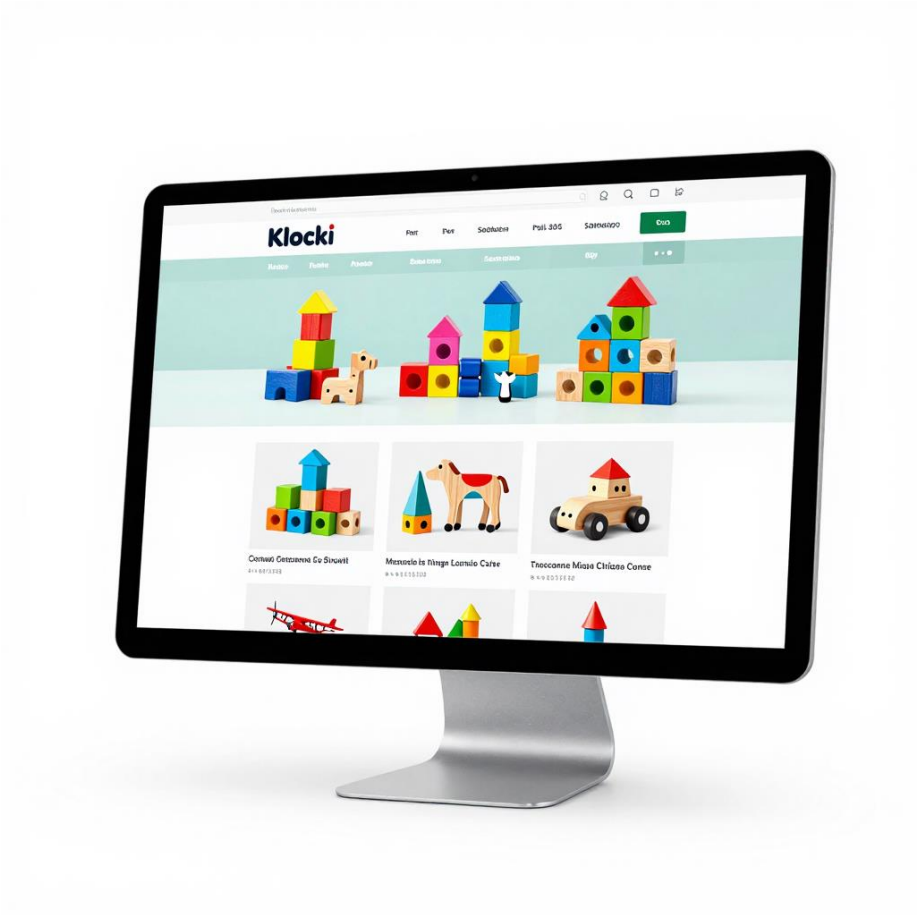
Wykład 7. Programowanie funkcyjne

- Strumienie
- Funktory
- Interfejsy funkcyjne
- Funkcje lambda

Wykład 8. Programowanie generyczne

- Typ Object
- Szablony klas i metod

Wykład 9. Pakiety



- Podział funkcjonalny aplikacji
- Tworzenie i import bibliotek
- Repozytoria pakietów

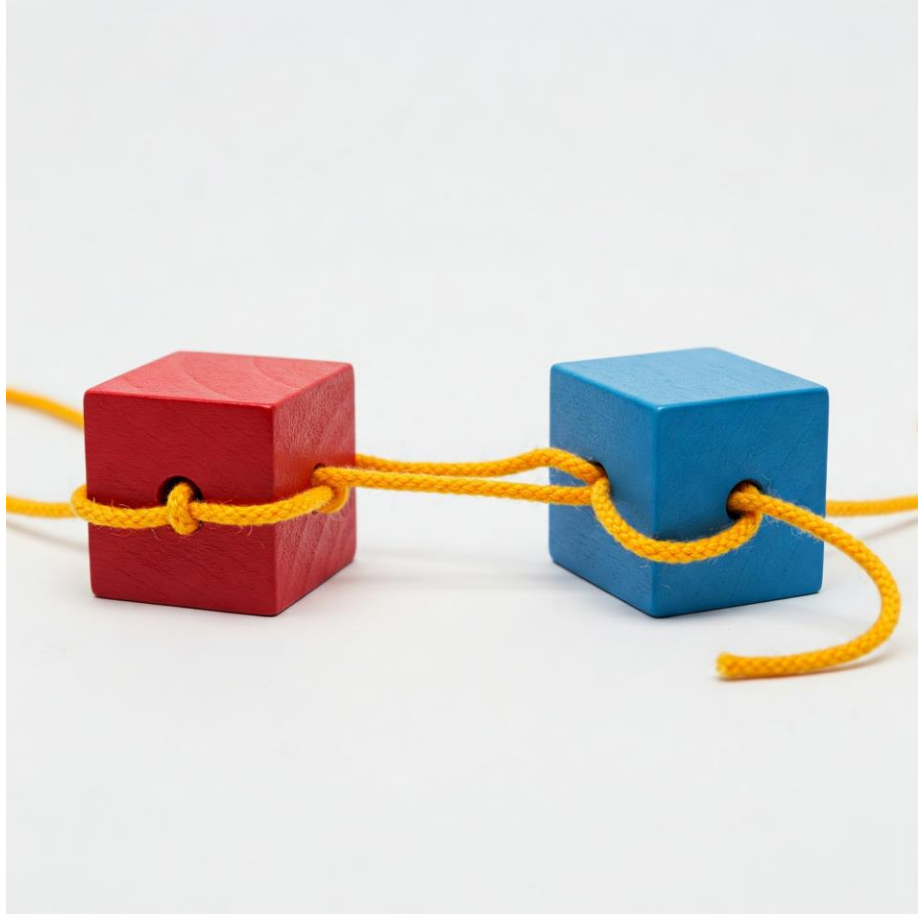
Wykład 10. Podstawy wdrożeń

- Deployment
- Testowanie
- Tworzenie dokumentacji

Wykład 11. Współbieżność

- Wątki
- Wywołania asynchroniczne

Wykład 12. Komunikacja sieciowa



- Gniazda sieciowe
- Asynchroniczny serwer
- Protokoły TCP i UDP
- Przesyłanie napisów i danych

Wykład 13. Interfejs graficzny

- Wybrane widgety Swing
- Obsługa zdarzeń

Wykład 14. Projektowanie interfejsu

- Komunikacja z aplikacją
- Ograniczenia fizyczne i poznawcze
- Elementy składowe i cechy dobrego interfejsu

Zaliczenie - laboratorium

Kolokwium po 6. temacie – 50 punktów.

Kolokwium po 13. temacie – 50 punktów.

Punkty z zajęć - 10 punktów.

Skala ocen:

2 - [0, 50],
3 - (50, 60],
3+ - (60, 70],
4 - (70, 80],
4+ - (80, 90],
5 - (90, 110].

Zaliczenie - egzamin

Forma pisemna, pytania teoretyczne.

Wynik z laboratorium +/- 15 punktów.

Zwolnienie od 95 punktów z laboratorium.

Skala ocen:

2 - [0, 50],
3 - (50, 60],
3+ - (60, 70],
4 - (70, 80],
4+ - (80, 90],
5 - (90, 110].

Zaliczenie - poprawa

Dwuetapowe:

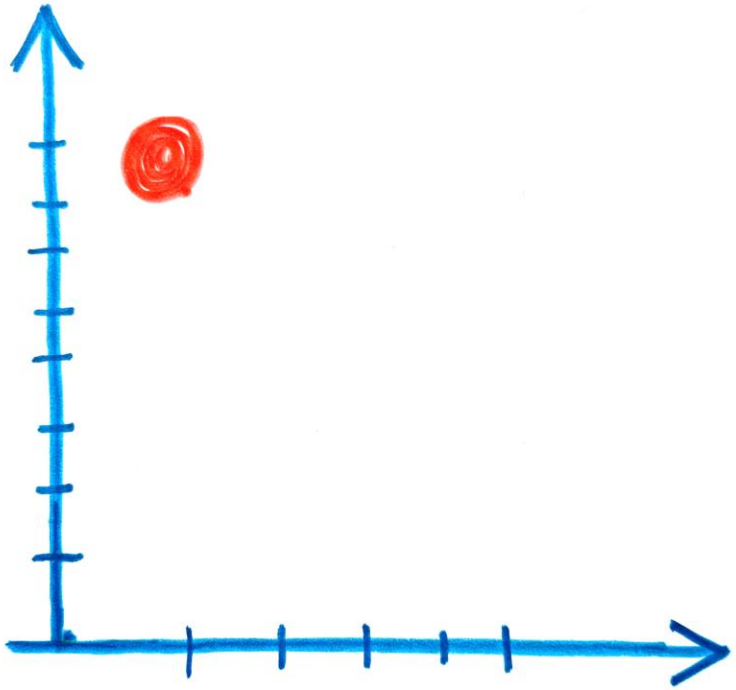
- pytania teoretyczne na kartce,
- program na komputerze.

Skala ocen:

2 - [0, 50],
3 - (50, 60],
3+ - (60, 70],
4 - (70, 80],
4+ - (80, 90],
5 - (90, 100].

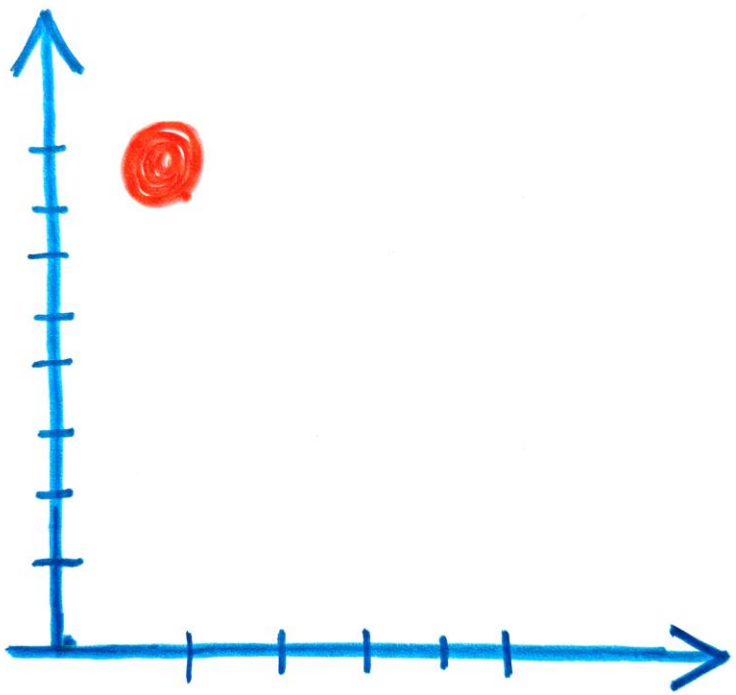
Struktury i klasy

Struktura



```
struct Point {
  double x;
  double y;
};
```

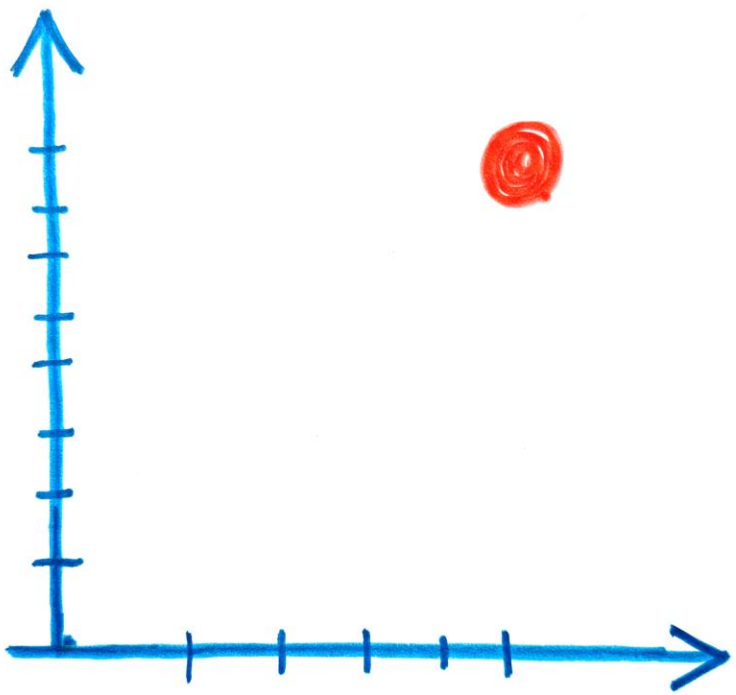
Obiekt strukturalny



```
struct Point {
  double x;
  double y;
};

Point p = {1.0, 7.5};
std::cout << p.x << " " << p.y << std::endl;
```

Bezpośredni dostęp do pól obiektu

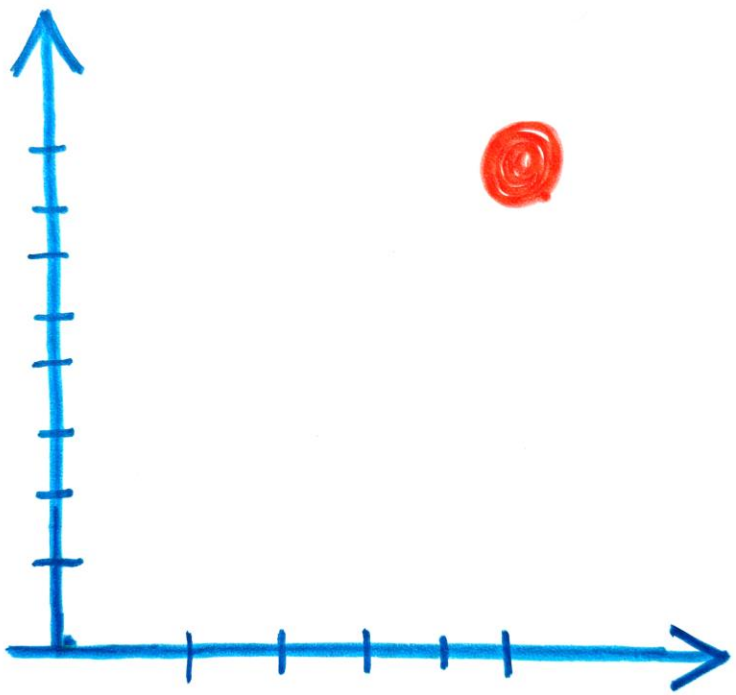


```
struct Point {
  double x;
  double y;
};

Point p = {1.0, 7.5};
std::cout << p.x << " " << p.y << std::endl;


p.x += 5.0;
std::cout << p.x << " " << p.y << std::endl;
```

Funkcje operujące na obiekcie



```
struct Point {
  double x, y;
};

void move(Point &p, double dx, double dy) {
  p.x += dx; p.y += dy;
}

void print(Point &p) {
  std::cout << p.x << " " << p.y << std::endl;
}

Point p = {1.0, 7.5};
move(p, 5.0, 0.0);
print(p);
```

Znajdź trzy różnice

```
struct Point {
  double x, y;


  void move(double dx, double dy) {
    x += dx; y += dy;
  }

  void print() {
    std::cout << x << " " << y << std::endl;
  }
};

Point p = {1.0, 7.5};
p.move(5.0, 0.0);
p.print();
```

```
struct Point {
  double x, y;
};

void move(Point &p, double dx, double dy) {
  p.x += dx; p.y += dy;
}

void print(Point &p) {
  std::cout << p.x << " " << p.y << std::endl;
}


Point p = {1.0, 7.5};
move(p, 5.0, 0.0);
print(p);
```

Znajdź ~~trzy~~ 21 różnic~~e~~

```cpp
struct Point {
  double x, y;

  void move(double dx, double dy) {
    x += dx; y += dy;
  }

  void print() {
    std::cout << x << " " << y << std::endl;
  }
};

Point p = {1.0, 7.5};
p.move(5.0, 0.0);
p.print();
```

```cpp
struct Point {
  double x, y;
};

void move(Point &p, double dx, double dy) {
  p.x += dx; p.y += dy;
}

void print(Point &p) {
  std::cout << p.x << " " << p.y << std::endl;
}

Point p = {1.0, 7.5};
move(p, 5.0, 0.0);
print(p);
```

Struktura obejmuje metody

```
struct Point {
  double x, y;

  void move(double dx, double dy) {
    x += dx; y += dy;
  }

  void print() {
    std::cout << x << " " << y << std::endl;
  }
};

Point p = {1.0, 7.5};
p.move(5.0, 0.0);
p.print();
```

```
struct Point {
  double x, y;
};

void move(Point &p, double dx, double dy) {
  p.x += dx; p.y += dy;
}

void print(Point &p) {
  std::cout << p.x << " " << p.y << std::endl;
}

Point p = {1.0, 7.5};
move(p, 5.0, 0.0);
print(p);
```

Metody widzą swój obiekt

```
struct Point {
  double x, y;

  void move(double dx, double dy) {
    x += dx; y += dy;
  }

  void print() {
    std::cout << x << " " << y << std::endl;
  }
};

Point p = {1.0, 7.5};
p.move(5.0, 0.0);
p.print();
```

```
struct Point {
  double x, y;
};

void move(Point &p, double dx, double dy) {
  p.x += dx; p.y += dy;
}

void print(Point &p) {
  std::cout << p.x << " " << p.y << std::endl;
}

Point p = {1.0, 7.5};
move(p, 5.0, 0.0);
print(p);
```

Metody wywoływane są na rzecz obiektu

```
struct Point {
  double x, y;

  void move(double dx, double dy) {
    x += dx; y += dy;
  }

  void print() {
    std::cout << x << " " << y << std::endl;
  }
};

Point p = {1.0, 7.5};
p.move(5.0, 0.0);
p.print();
```

```
struct Point {
  double x, y;
};

void move(Point &p, double dx, double dy) {
  p.x += dx; p.y += dy;
}

void print(Point &p) {
  std::cout << p.x << " " << p.y << std::endl;
}

Point p = {1.0, 7.5};
move(p, 5.0, 0.0);
print(p);
```

Struktura to wyłącznie dane

```
struct Point {
  double x, y;
};
```

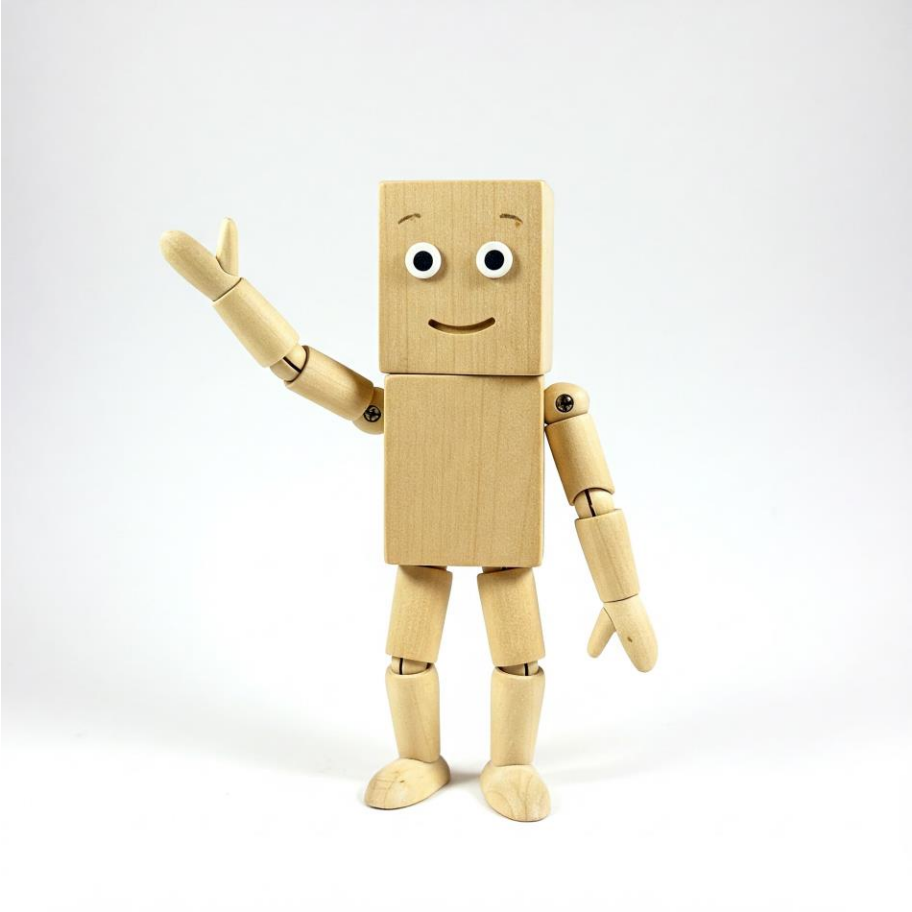
Funkcje używają danych ze struktury

```
struct Point {
  double x, y;
};

void move(Point &p, double dx, double dy) {
  p.x += dx; p.y += dy;
}

void print(Point &p) {
  std::cout << p.x << " " << p.y << std::endl;
}
```

Metody są działaniami w ramach struktur



```
struct Point {
  double x, y;

  void move(double dx, double dy) {
    x += dx; y += dy;
  }

  void print() {
    std::cout << x << " " << y << std::endl;
  }
};
```

Struktury w C, a struktury w C++

```c
struct Point {
  double x, y;
};

void move(struct Point *p, double dx, double dy) {
  p->x += dx;
  p->y += dy;
}

void print(struct Point *p) {
  printf("%f %f\n", p->x, p->y) ;
}

struct Point p = {1.0, 7.5};
move(&p, 5.0, 0.0);
print(&p);
```

```cpp
struct Point {
  double x, y;

  void move(double dx, double dy) {
    x += dx; y += dy;
  }

  void print() {
    std::cout << x << " " << y << std::endl;
   }
};

Point p = {1.0, 7.5};
p.move(5.0, 0.0);
p.print();
```

Struktury, a klasy w C++

```cpp
struct Point {

  double x, y;

  void move(double dx, double dy) {
    x += dx; y += dy;
  }

  void print() {
    std::cout << x << " " << y << std::endl;
  }
};

Point p = {1.0, 7.5};
p.move(5.0, 0.0);
p.print();
```

=

```cpp
class Point {
public:
  double x, double y;

  void move(double dx, double dy) {
    x += dx; y += dy;
  }

  void print() {
    std::cout << x << " " << y << std::endl;
  }
};

Point p = {1.0, 7.5};
p.move(5.0, 0.0);
p.print();
```

Klasy w Javie

```cpp
class Point {
public:
  double x, y;

  void move(double dx, double dy) {
    x += dx; y += dy;
  }

  void print() {
    std::cout << x << " " << y << std::endl;
  }
};

Point p = {1.0, 7.5};
p.move(5.0, 0.0);
p.print();
```

```java
public class Point {

  public double x, y;

  public void move(double dx, double dy) {
    x += dx; y += dy;
  }

  public void print() {
    System.out.println(x + " " + y);
  }
}

Point p = new Point();
p.x = 1.0; p.y = 7.5;
p.move(5.0, 0.0);
p.print();
```

Java

Rozwój Javy

- **1990** – Początek prac
- **1993** – Powstanie apletów
- **1995** – "Java"
- **1996** – JDK 1.0
- **1998** – SE, EE, ME
- **2004** – Scala
- **2007** – OpenJDK
- **2008** – Android
- **2010** – Przejęcie Sun przez Oracle
- **2011** – Kotlin
- **2014** – Java SE 8 LTS
- **2018** – Język #2 wg. PYPL

Budowanie i uruchamianie

Budowanie i uruchamianie

Java Runtime Environment

Środowisko uruchomieniowe, zawiera:

- maszynę wirtualną,
- biblioteki standardowe.

Java Development Kit

Środowisko deweloperskie, zawiera:

- środowisko uruchomieniowe,
- narzędzia deweloperskie:
  - kompilator,
  - debugger,
  - archiwizer,
  - ...

"Hello world!"

```cpp
#include <iostream>
int main(int argc, char *argv[]) {
  std::cout << "Hello World!" << std::endl;
  return 0;
}
```

```java
public class Main {
    public static void main(String[] args){
        System.out.printf("Hello world!");
    }
}
```

Pełna obiektowość

```cpp
#include <iostream>
int main(int argc, char *argv[]) {
  std::cout << "Hello World!" << std::endl;
  return 0;
}
```

```java
public class Main {
    public static void main(String[] args){
        System.out.printf("Hello world!");
    }
}
```

prawie

Pełna obiektowość

```cpp
#include <iostream>
int main(int argc, char *argv[]) {
  std::cout << "Hello World!" << std::endl;
  return 0;
}
```

```java
public class Main {
    public static void main(String[] args){
        System.out.printf("Hello world!");
    }
}
```

Typy proste

C++	Java	Rozmiar (bajty)
char	byte	1
wchar_t	char	2
short	short	2
int	int	4
long/long int	long	8
float	float	4
double	double	8
bool	boolean	1

C++ - bez odpowiednika w Javie

unsigned char

unsigned short int

unsigned int

unsigned long int

size_t

Inicjalizacja zmiennej typu prostego

```cpp
int unknown;
// unknown == ???
int variable = 10;
// variable == 10

int copy = variable;
// variable == 10, copy == 10
copy = 5;
// variable == 10, copy == 5

int &reference = variable;
// variable == 10, reference == 10
reference = 5;
// variable == 5, reference == 5

int *pointer = new int(10);
// *pointer == 10
```

```java
int known;
//known == 0
int variable = 10;
// variable == 10

int copy = variable;
// variable == 10, copy == 10
copy = 5;
// variable == 10, copy == 5
```

Inicjalizacja obiektu

```cpp
struct Point {
  double x, y;
};

Point p;
p.x = 1.0;
p.y = 7.5;
```

```java
public class Point {
  public double x, y;
}

Point p;
p.x = 1.0;
p.y = 7.5;
```

Inicjalizacja obiektu

```
struct Point {
  double x, y;
};

Point p;
p.x = 1.0;
p.y = 7.5;
```

obiekt

```
public class Point {
  public double x, y;
}

Point p;
p.x = 1.0;
p.y = 7.5;
```

referencja

Inicjalizacja obiektu

```cpp
struct Point {
  double x, y;
};

Point p;
p.x = 1.0;
p.y = 7.5;
```

```java
public class Point {
  public double x, y;
}

Point p = null;
p.x = 1.0;
p.y = 7.5;
```

Inicjalizacja obiektu

```cpp
struct Point {
  double x, y;
};

Point* p = new Point;
p->x = 1.0;
p->y = 7.5;

delete p;
```

```java
public class Point {
  public double x, y;
}

Point p = new Point();
p.x = 1.0;
p.y = 7.5;
```

referencja

obiekt

Usunięcie obiektu

```cpp
struct Point {
  double x, y;
};

Point* p = new Point;
p->x = 1.0;
p->y = 7.5;

delete p;
```

```java
public class Point {
  public double x, y;
}

Point p = new Point();
p.x = 1.0;
p.y = 7.5;
```

Automatyczne zarządzanie pamięcią

- Zabezpieczenie przed wyciekiem pamięci.
- Usuwanie obiektów pozbawionych referencji.
- Garbage collector - niezależny i niekontrolowalny proces.